

Apuntes de Clase

IC-6200 Inteligencia Artificial

Kevin Jiménez Molinares
Instituto Tecnológico de Costa Rica
Profesor: Steven Pacheco Portugués

Index Terms—LLM, tokenización, embeddings, espacio vectorial, similitud del coseno, RAG, inteligencia artificial.

I. DUDAS Y NOTICIAS

- **Consulta por limitaciones en el notebook:** Se consultó que debido al uso de herramientas en la tarea estas no se pudieran ejecutar en el notebook como el caso de **Hydra**, a esto el profe recomendó utilizar archivos .py y hacer una ejecución fuera de notebook.
- **Consulta con la arquitectura del Autoencoder:** Se valida la adaptación del modelo propuesto siempre y cuando se mantenga la latencia.
- **Google I/O:** Se conversó sobre este evento que representa una transformación hacia una era de agentes de IA, durante este evento se anunciaron nuevas salidas o versiones de modelos como Gemini 3.5 Flash entre muchos otros mas.

II. PANORAMA GENERAL DE LOS LLMs

Los Modelos de Lenguaje Grandes se han convertido en una pieza central de la inteligencia artificial porque pueden entender y generar texto, además de interactuar con código, imágenes y audio, su utilidad no se limita a responder preguntas, también sirven para resumir, traducir, clasificar, etc. En términos prácticos, un LLM aprende a predecir el siguiente token a partir del contexto previo. Esa aparente sencillez es la base de comportamientos mucho más ricos, como la redacción fluida, la resolución de problemas y la adaptación al estilo del usuario, su potencia trae consigo limitaciones importantes como por ejemplo pueden inventar información, olvidan el contexto cuando este supera su ventana de entrada y no poseen conocimiento actualizado por sí mismos.

III. ESTRUCTURA BÁSICA DE UNA RED NEURONAL

Para entender cómo trabaja un LLM por dentro, conviene recordar los componentes esenciales de una red neuronal. A continuación se enumeran cada parte con una interpretación sencilla:

- **Entradas (Inputs):** Datos iniciales que recibe el modelo para comenzar el procesamiento.
- **Pesos (Weights):** Valores que determinan qué tanto influye cada entrada en el resultado.
- **Nodos ocultos:** Unidades intermedias que combinan información, aplican transformaciones y detectan patrones.
- **Salida (Output):** Respuesta final producida por la red, por ejemplo una predicción o clasificación.

El flujo de información avanza capa por capa. Cada capa transforma la representación previa hasta obtener una salida útil.

IV. TOKENIZACIÓN

Antes de que un modelo pueda trabajar con lenguaje natural, el texto debe convertirse en una secuencia de unidades discretas llamadas *tokens*. Un token puede ser una palabra completa, una subpalabra, un fragmento de palabra o incluso un carácter.

La tokenización tiene dos objetivos principales: reducir el texto a una forma manejable para el sistema y conservar, en la medida de lo posible, la información lingüística relevante. En implementaciones reales, es común normalizar el texto, por ejemplo pasando a minúsculas o segmentando según reglas estadísticas aprendidas a partir de grandes corpus.

IV-A. Tipos habituales de tokenización

A continuación se integran las variantes más frecuentes y su utilidad general:

- **Por palabra:** Sencilla de entender, aunque produce vocabularios enormes.
- **Por carácter:** Reduce el problema de palabras desconocidas, pero pierde estructura semántica de largo alcance.
- **Subpalabra:** Equilibra tamaño de vocabulario y capacidad de generalización; es una de las opciones más usadas hoy.
- **Byte-level:** Trabaja con bytes, por lo que soporta cualquier idioma, símbolo o emoji.

V. EMBEDDINGS Y ESPACIO VECTORIAL

Los modelos transforman los tokens en *embeddings*, es decir, vectores densos dentro de un espacio matemático de alta dimensión. A diferencia de los IDs simples, los embeddings sí capturan relaciones semánticas y contextuales.

V-A. Representación conceptual

Cuando dos palabras tienen significados parecidos, sus vectores tienden a ubicarse cerca. Por ejemplo, conceptos como *casa* y *hogar* suelen aparecer en regiones similares del espacio vectorial. Lo mismo ocurre con relaciones más abstractas, como género, jerarquía, tiempo o dominio temático.

En una versión simplificada, el modelo puede operar con relaciones tipo:

$$\mathbf{v}(\text{Rey}) - \mathbf{v}(\text{Hombre}) + \mathbf{v}(\text{Mujer}) \approx \mathbf{v}(\text{Reina}) \quad (1)$$

Aunque esta ecuación no describe todas las complejidades del lenguaje, sí ilustra una idea central: los embeddings permiten hacer aritmética con conceptos.

V-B. Similitud semántica

Una vez que las palabras se convierten en vectores, es posible medir su cercanía. Las métricas más comunes son la distancia euclidiana y la similitud del coseno. La primera evalúa la separación en línea recta entre dos puntos; la segunda compara el ángulo entre vectores y suele ser más útil en procesamiento de lenguaje natural porque prioriza la dirección sobre la magnitud. A continuación se resumen las más comunes:

- **Distancia euclidiana:** Mide qué tan lejos están dos vectores en línea recta.
- **Similitud del coseno:** Evalúa cuánto se parecen dos vectores según el ángulo entre ellos.

Las expresiones matemáticas más usadas son:

$$d(a, b) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2} \quad (2)$$

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \|b\|} \quad (3)$$

Cuando la similitud del coseno es alta, los vectores apuntan en direcciones parecidas y, por tanto, los términos suelen compartir contexto o intención.

VI. CAPACIDADES EMERGENTES DE LOS LLMs

Gracias al entrenamiento masivo y a la arquitectura Transformer, los LLMs desarrollan habilidades que no fueron programadas explícitamente una por una. Estas capacidades emergen del aprendizaje estadístico sobre grandes cantidades de datos. A continuación se enumeran las capacidades emergentes más relevantes:

- **Comprensión contextual:** Interpretan el significado según las oraciones vecinas.
- **Generación coherente:** Producen textos fluidos con estilo consistente.
- **Razonamiento:** Resuelven tareas lógicas o explican procesos paso a paso.
- **In-context learning:** Ajustan su comportamiento a partir de ejemplos dentro del mismo chat.
- **Multitarea:** Pueden redactar, clasificar, dialogar y programar sin cambiar de arquitectura.

Estas habilidades explican por qué un solo modelo puede usarse en tantos escenarios distintos. Aun así, su desempeño depende en gran medida de la calidad del contexto y de la forma en que se formulen las instrucciones.

VII. LIMITACIONES PRINCIPALES

Junto con sus ventajas, los LLMs presentan fallos que no se pueden ignorar en entornos reales. A continuación se resumen las limitaciones más importantes:

- **Alucinaciones:** El modelo puede inventar datos incorrectos con gran seguridad.

- **Memoria limitada:** Solo conserva el contexto que cabe en la ventana de entrada.
- **Conocimiento estático:** No se actualiza solo con eventos recientes.
- **Costo computacional:** Su entrenamiento y despliegue requieren infraestructura costosa.

Estas restricciones son especialmente problemáticas cuando se necesita precisión factual, datos empresariales internos o información que cambia con frecuencia. Por esa razón, en muchas aplicaciones no basta con usar el LLM de forma aislada.

VIII. ARQUITECTURA RAG

La arquitectura *Retrieval-Augmented Generation* (RAG) aparece como respuesta práctica a dos problemas centrales: las alucinaciones y el conocimiento estático. En lugar de pedirle al modelo que responda solo con lo que aprendió durante el entrenamiento, se le conecta a una fuente externa de información que puede consultarse en tiempo real.

RAG trabaja mejor cuando se divide en dos grandes momentos: una etapa de pre-producción, donde se prepara el conocimiento, y una etapa de producción, donde el sistema responde a las preguntas de los usuarios.

VIII-A. Ingesta, limpieza y fragmentación

En la fase inicial, los documentos deben reunirse, depurarse y dividirse en fragmentos más pequeños llamados *chunks*. Esta fragmentación suele hacerse entre 200 y 500 tokens, con cierto solapamiento para evitar que una idea importante quede cortada entre dos segmentos. Luego, cada fragmento se transforma en un embedding y se guarda en una base de datos vectorial junto con su metadata. A continuación se enumeran las etapas de preparación en RAG:

- **Preparación:** Se reúnen y limpian los documentos o fuentes de datos.
- **Chunking:** El texto largo se divide en fragmentos independientes.
- **Vectorización:** Cada fragmento se convierte en un vector semántico.
- **Indexación:** Los vectores y su metadata se almacenan para búsqueda rápida.

VIII-B. Recuperación semántica

Cuando el usuario hace una consulta, esa pregunta también se convierte en un embedding. Después, el sistema calcula su similitud contra los vectores almacenados y recupera los fragmentos más cercanos. La ventaja de este enfoque es que la búsqueda no depende solo de palabras exactas, sino del significado de la consulta.

VIII-C. Aumentación y generación

Una vez recuperada la evidencia relevante, esta se inserta en un prompt estructurado. El LLM recibe entonces una instrucción como: usar únicamente el contexto proporcionado para responder. De esta manera, la salida queda anclada a

información recuperada y no a una inferencia libre del modelo. En aplicaciones reales, esto mejora la precisión, reduce alucinaciones y permite trabajar con conocimiento privado o actualizado.